INF266 - Problem Set 5

Ragnhild Klette, Maya Evensen, Halvard Haavardsruud

April 7, 2026

Part I - Problems

Exercise 1 - Stocastic policies

1.

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

States

$$\mathcal{S} = \{s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7\}$$

$$\begin{aligned}
&s_0 : \text{empty}, \quad s_1 : \text{full}, \quad s_2 : \text{sugared}, \quad s_3 : \text{salted}, \\
&s_4 : \text{sugar sampling}, \quad s_5 : \text{salt sampling}, \\
&s_6 : \text{baked sugary}, \quad s_7 : \text{baked salty}
\end{aligned}$$

Actions

$$\begin{aligned}
&\mathcal{A}(s_0) = \{\text{wait}, \text{prep}\}, \\
&\mathcal{A}(s_1) = \{\text{wait}, \text{sugar}, \text{salt}\}, \\
&\mathcal{A}(s_2) = \mathcal{A}(s_3) = \{\text{taste}, \text{cook}, \text{restart}\}, \\
&\mathcal{A}(s_4) = \mathcal{A}(s_5) = \{\text{cook}, \text{return}\}, \\
&\mathcal{A}(s_6) = \mathcal{A}(s_7) = \emptyset
\end{aligned}$$

Transitions (deterministic)

$$P(s'|s,a) = 1 \text{ for:}$$

$$\begin{aligned}
&(s_0, \text{wait}) \to s_0, \ (s_0, \text{prep}) \to s_1 \\
&(s_1, \text{sugar}) \to s_2, \ (s_1, \text{salt}) \to s_3 \\
&(s_2, \text{taste}) \to s_4, \ (s_2, \text{cook}) \to s_6, \ (s_2, \text{restart}) \to s_1 \\
&(s_3, \text{taste}) \to s_5, \ (s_3, \text{cook}) \to s_7, \ (s_3, \text{restart}) \to s_1 \\
&(s_4, \text{cook}) \to s_6, \ (s_4, \text{return}) \to s_2 \\
&(s_5, \text{cook}) \to s_7, \ (s_5, \text{return}) \to s_3
\end{aligned}$$

Reward

$$R(s,a,s') = \begin{cases} +10 & s' = s_6 \\ -10 & s' = s_7 \\ 0 & \text{otherwise} \end{cases}$$

$$\gamma \in [0,1]$$

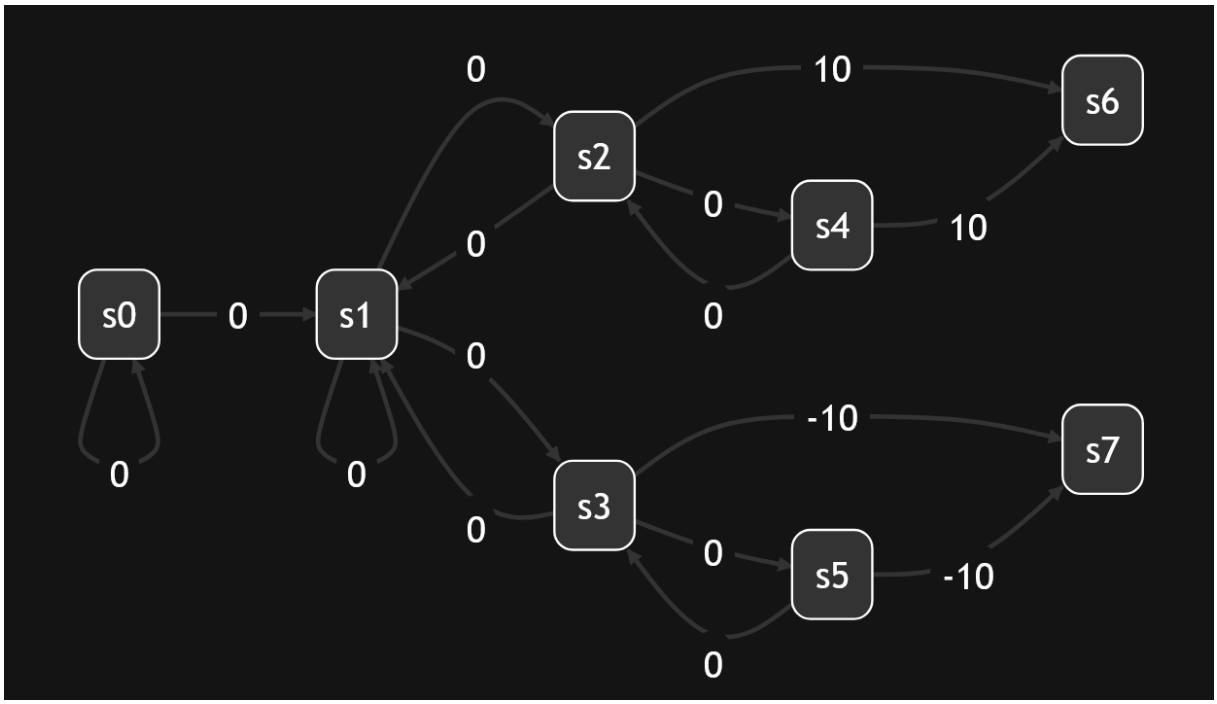

Figure 1: Backup diagram for the MDP.

2.

3. (*) **Y**es, we can define an optimal deterministic policy for this MDP. The optimal policy would be:

   (a) In state s_0, take action prep to move to state s_1.

   (b) In state s_1, take action sugar to move to state s_2.

   (c) In state s_2, take action cook to move to state s_6 and receive a reward of $+10$.

   This policy is optimal because it leads to the highest possible reward of $+10$ by reaching state s_6. Any deviation from this policy would either lead to a lower reward (e.g., going to state s_7 with a reward of -10), a delayed reward (like taking action taste in s_2 leading to state s_4), or no reward at all (e.g., waiting in state s_0 or taking the restart action in states s_2 or s_3). Therefore, the optimal deterministic policy is to always choose the actions that lead to state s_6. This policy is simply the shortest path to the highest reward.

4. (*) **Y**es, an optimal stochastic policy exists. However, since the environment is fully deterministic and there is a unique optimal trajectory leading to reward $+10$, any randomness that assigns non-zero probability to suboptimal actions (e.g., adding salt) will reduce the expected return. Thus, the optimal stochastic policy is actually just deterministic, meaning the agent always chooses the best action with probability 1 and never chooses any other actions.

$$\pi(a \mid s) = \begin{cases} 1 & \text{if } a = a^*(s), \\ 0 & \text{otherwise.} \end{cases}$$

   In particular, the agent always chooses actions that lead to the sugary cookies and avoids all actions leading to the salty outcome.

5. We define the MDP as

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma).$$

   States To retain the Markov property, we include the latent variable $\sigma \in \{\text{sugar}, \text{salt}\}$:

$$\mathcal{S} = \{s_0, s_1, (s_2, \sigma), s_4, s_5, s_6, s_7\},$$

where

$$
\begin{aligned}
s_0 &: \text{empty bowl}, \quad s_1 : \text{full bowl}, \\
(s_2, \sigma) &: \text{enriched bowl with ingredient } \sigma, \\
s_4 &: \text{sugar sampling}, \quad s_5 : \text{salt sampling}, \\
s_6 &: \text{baked sugary cookies}, \quad s_7 : \text{baked salty cookies}.
\end{aligned}
$$

Actions

$$
\begin{aligned}
\mathcal{A}(s_0) &= \{\text{wait}, \text{prepare dough}\}, \\
\mathcal{A}(s_1) &= \{\text{wait}, \text{add ingredient}\}, \\
\mathcal{A}((s_2, \sigma)) &= \{\text{taste}, \text{cook}, \text{restart}\}, \\
\mathcal{A}(s_4) = \mathcal{A}(s_5) &= \{\text{cook}, \text{return}\}, \\
\mathcal{A}(s_6) = \mathcal{A}(s_7) &= \emptyset.
\end{aligned}
$$

Transitions

$$
\begin{aligned}
&(s_0, \text{wait}) \to s_0, \quad (s_0, \text{prepare dough}) \to s_1, \\
&(s_1, \text{add ingredient}) \to \begin{cases} (s_2, \text{sugar}) & \text{with prob } 0.5, \\ (s_2, \text{salt}) & \text{with prob } 0.5, \end{cases} \\
&((s_2, \sigma), \text{taste}) \to \begin{cases} s_4 & \text{if } \sigma = \text{sugar}, \\ s_5 & \text{if } \sigma = \text{salt}, \end{cases} \\
&((s_2, \sigma), \text{cook}) \to \begin{cases} s_6 & \text{if } \sigma = \text{sugar}, \\ s_7 & \text{if } \sigma = \text{salt}, \end{cases} \\
&((s_2, \sigma), \text{restart}) \to \begin{cases} (s_2, \text{sugar}) & \text{with prob } 0.5, \\ (s_2, \text{salt}) & \text{with prob } 0.5, \end{cases} \\
&(s_4, \text{cook}) \to s_6, \quad (s_4, \text{return}) \to (s_2, \text{sugar}), \\
&(s_5, \text{cook}) \to s_7, \quad (s_5, \text{return}) \to (s_2, \text{salt}).
\end{aligned}
$$

Reward function

$$
R(s, a, s') = \begin{cases} +10 & \text{if } s' = s_6, \\ -10 & \text{if } s' = s_7, \\ 0 & \text{otherwise}. \end{cases}
$$

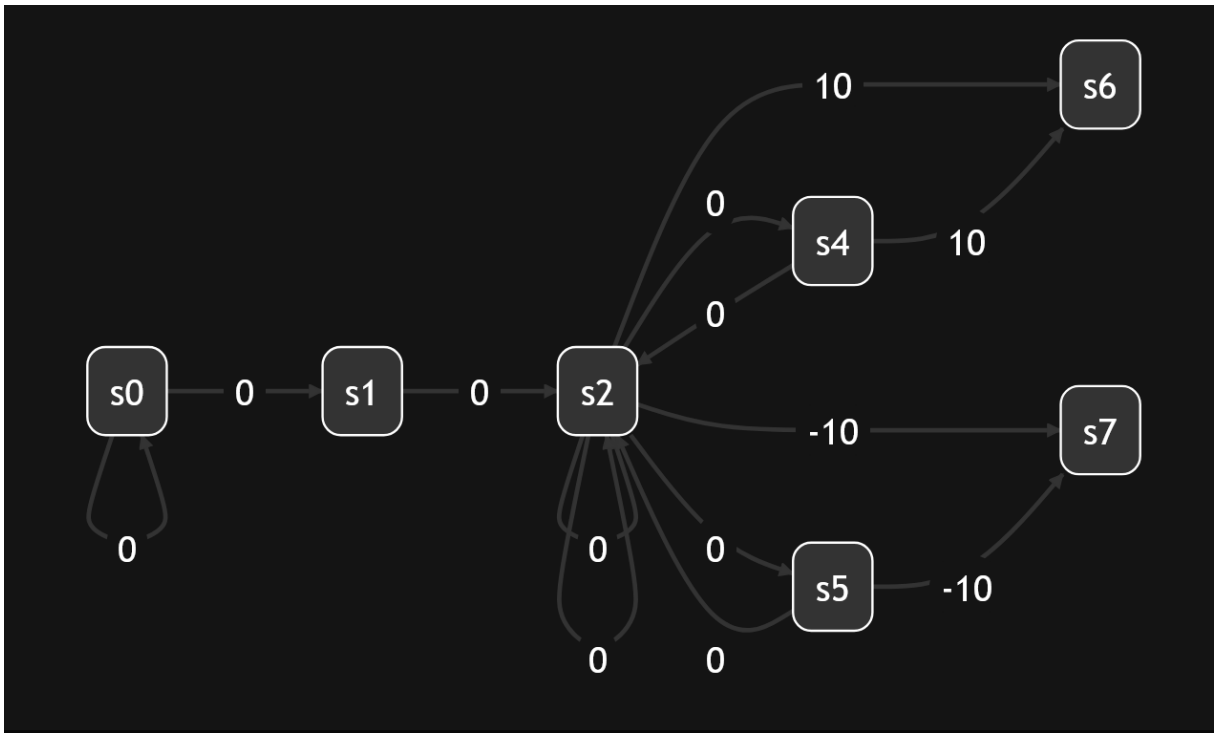

Figure 2: Backup diagram for the new MDP.

6.

7. (*) No, an optimal deterministic policy does not exist. The agent cannot observe the latent variable $\sigma \in \{\text{sugar}, \text{salt}\}$, so in the enriched bowl state it cannot distinguish between the two underlying cases. A deterministic policy must choose the same action regardless of σ. However, the optimal behavior depends on information: the agent should first *taste* to reveal σ, and then choose *cook* if $\sigma = \text{sugar}$ or *restart* if $\sigma = \text{salt}$. This requires conditioning actions on information that is not available in the current observation. Thus, no deterministic memoryless policy can achieve optimal performance. An optimal policy must either use memory (history-dependent) or be stochastic.

8. (*) Yes, an optimal stochastic policy exists.

   Due to partial observability, the agent must act under uncertainty in the enriched bowl state. The optimal strategy is to first gather information and then act optimally based on that information.

   An optimal policy is:

   $$\pi(\text{taste} \mid \text{enriched bowl}) = 1,$$

   and then

   $$\pi(\text{cook} \mid \text{sugar sampling}) = 1, \quad \pi(\text{return} \mid \text{salt sampling}) = 1.$$

   After returning to the enriched bowl from salt sampling, the agent again follows:

   $$\pi(\text{taste} \mid \text{enriched bowl}) = 1.$$

   Thus, the agent repeatedly tastes until it identifies sugar, and only then chooses to cook, ensuring the maximal expected reward.

9. (*) The key difference between the original MDP and the new MDP is that the new MDP introduces partial observability through the latent variable σ that determines whether the ingredient added is sugar or salt. In the original MDP, the agent has full observability and can directly choose actions based on the current state. In contrast, in the new MDP, the agent cannot directly observe σ and must rely on actions (like tasting) to gain information about it. This means that an optimal policy may require memory or stochasticity, as the agent needs to condition its actions based on the information gathered.

10. Yes, a natural modification to the original MDP is to make the latent variable σ observable, but only after taking a specific action (e.g., tasting). This creates a scenario where the agent must first take an exploratory action to gain information about the state of the environment before it can make an informed decision. This would restore full observability while still requiring the agent to gather information. Alternativly we can augment the state space to incude the history of actions take, allowing the agent to infer the latent variable from past actions and observations. In both cases we remove partial observability and simplify the policy design.

Exercise 2 - Continuous actions

1. The policy $\pi(a \mid s)$ is parametrized using a neural network that takes the state s as input and outputs a vector of real values, one for each action:

$$z(s) = (z_1(s), \ldots, z_n(s)).$$

These outputs are then converted into a probability distribution over actions using the softmax function:

$$\pi(a_i \mid s) = \frac{\exp(z_i(s))}{\sum_{j=1}^{n} \exp(z_j(s))}.$$

This ensures that the policy defines a valid probability distribution over the discrete action space.

2. For a continuous action space $\mathcal{A} = \mathbb{R}$, we can model the policy as a gaussian (normal) distribution.

$$\pi(a \mid s) = \mathcal{N}(a \mid \mu(s), \sigma^2(s)).$$

The mean $\mu(s)$ and variance $\sigma^2(s)$ are given by a neural network as functions of the state s.

This allows the policy to output a continuous distribution over actions while remaining differentiable and suitable for gradient-based optimization.

3. As previously stated we would need mean $\mu(s)$ and variance $\sigma^2(s)$ to model the policy for a gaussian distribution.

4. (*) To define a policy over $\mathbb{R}$, we parametrize a Gaussian distribution whose parameters are given by a neural network.

The network takes the state s as input and outputs two values:

$$\mu(s), \quad \log \sigma(s).$$

These define the policy as stated in task 2. To ensures that $\sigma(s) > 0$ we can exponentiate the output for the variance:

$$\sigma(s) = \exp(\log \sigma(s)).$$

5. The choise of a gaussian distribution assumes that actions are continous and can be represented by a single mean and variance. This means that we also assume symmetry in the deviations and that the action space is unbounded. We also assume that the distribution is unimodal, meaning that there is a single peak leading to a single optimal action for each state.

Exercise 3 - MAB and Baselines

1. (*) The term $\hat{E}[R]$ plays the role as the baseline. In the update rule, the quantity $(R^{(t)} - \hat{E}[R])$ represents how much better or worse the received reward is compared to the average reward. This is analogous to the baseline used in policy gradient methods.

2. (*) This term serves as a baseline to calculate wether a given reward is better or worse than the average reward. If the value $(R^{(t)} - \hat{E}[R])$ is positive, it indicates that the action is better than average, and that we should increase the probability of taking that action. Contrary if the value is negative it indicates that the action is worse than average, and that we should decrease the probability of taking that action in the future. The increase or decrease scaled by the learning rate α shows how much we adjust the policy based on the observed reward compared to the baseline.

Exercise 4 - Actor-critic methods

1. In actor-critic methods, the critic correspond to policy evaluation and actor correspond to policy improvement. The critic estimates the value function $v^\pi(s)$ or $q^\pi(a, s)$, for the current policy. This is analogous to policy evaluation, where we assess how good the current policy is. The actor updates the policy parameters in the direction suggested by the critic. This is analogous to policy improvement, where the policy is adjusted to favor better actions. Thus, the critic provides feedback about the quality of actions, while the actor uses this feedback to improve the policy. In this sence an actor and a cretic can be assosiated with policy improvement and policy evaluation, respectively.

Part II - Report

Exercise 5 - Function approximation and policy gradient

1. This task is a continuous action problem, as the action space is $\mathcal{A} = \mathbb{R}$. There is also no terminal state, as the agent receives a reward at every time step and the environment does not have a natural endpoint, making this a continuing task.

2. We implemented Q-learning using a linear approximation of the action-value function, where

$$Q(s, a) = w^\top \phi(s, a),$$

with $\phi(s, a)$ representing a feature vector constructed from the state-action pair. The features include the robot position, velocity, cookie position, remaining time, and the relative distance between the robot and the cookie. Action-specific features are used so that each action has its own set of weights.

The agent was trained using an ϵ-greedy exploration strategy and updated according to the standard Q-learning rule based on the temporal-difference (TD) error:

$$w \leftarrow w + \alpha\big(r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big)\phi(s, a).$$

The results show that the average return increases from approximately 4 at the beginning of training to around 13 after convergence, indicating that the agent successfully learns a meaningful policy. In particular, the agent learns to move toward the cookie efficiently while partially controlling its velocity to avoid penalties.

However, the performance stabilizes below the optimal level. This is likely due to the limitations of the linear approximation, which cannot fully capture the non-linear dynamics of the environment, such as the effects of friction and velocity-dependent rewards.

4. (*) We implemented two value-based approaches to approximate $q(s, a)$: a linear model and a neural network.

The action-value function is approximated as a linear combination of features:

$$q(s, a) \approx \mathbf{w}_a^\top \phi(s).$$

This method is simple and stable. The results show steady improvement, with average return increasing from about 4 to around 13. However, performance plateaus, indicating limited representational capacity.

Neural Network Approximation. The neural network leads to higher performance, with average returns reaching around 16 and peaks above 19. Learning is initially unstable but improves over time.

The neural network preforms better than the linear model due to its greater expressive power, but it is less stable during training. The linear model is more stable but achieves lower final performance.

6. (*) We implemented an actor-critic method, where the actor learns a policy $\pi(a|s)$ and the critic estimates $V(s)$. The update is based on the TD error:

$$\delta = r + \gamma V(s') - V(s).$$

The method improves quickly from negative returns to stable performance around 17–18, with peaks above 23.

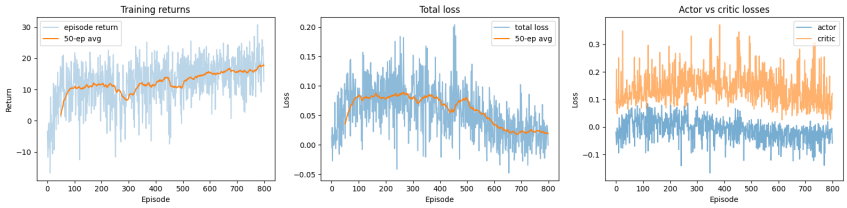

Figure 3: Traning preformance

Comparison. Compared to value-based methods:

- It outperforms the linear approximation,
- It achieves similar or slightly better performance than neural Q-learning,
- It produces smoother and more stable behavior, though with some variance.